

Automating IP Address Access Control for Secure Content

Role: Cybersecurity Intern Analyst

Tools: Python

Project Overview

At my current healthcare organization, secure access to sensitive content is managed through an allow list of authorized IP addresses. These IPs belong to employees who work directly with personal patient records. Over time, some IPs need to be removed due to role changes, departures, or access revocations. To ensure our security measures stay up-to-date, I developed a Python script that automatically updates the `allow_list.txt` file by removing any IPs found in a separate `remove_list`.

Read the Allow List File

I started by assigning the allow list filename to a variable and using a `with open()` block to safely read the contents of `allow_list.txt`. This ensures the file is automatically closed after it's read, minimizing the risk of file corruption or resource leaks.

```
# Define the name of the allow list file
import_file = "allow_list.txt"

# Open the allow list file in read mode and store its contents
with open(import_file, "r") as file:
    ip_addresses = file.read() # Read the file content as a single string
```

Convert String to List

After reading the file contents as a single string, I converted it into a list using the `.split()` method. This allows me to work with each IP address as a separate item.

```
#Use the .split() to convert ip addresses from a string to a list
ip_addresses = ip_addresses.split()
```

Define the Remove List

I defined a `remove_list` variable to store the IP addresses that should no longer have access. These are manually input or retrieved from another file, depending on the use case.

```
# Define a list of IP addresses that should be removed from the allow list
remove_list = ["192.168.0.12", "10.0.0.5"]
```

Remove IPs from the Allow List

Next, I used a `for` loop to go through the `remove_list`. For each IP, the script checks if it's in the current allow list. If it is, it's removed using the `.remove()` method. I added a check to prevent errors in case an IP in the `remove_list` doesn't exist in the allow list.

```
# Iterate through the remove list and remove matching IPs from the allow list
for ip in remove_list:
    if ip in ip_addresses:
        ip_addresses.remove(ip)
```

Update the Allow List File

Once the list was updated, I converted it back to a string using `.join()`, placing each IP on a new line. Then, I used another `with open()` block in write mode to overwrite the existing `allow_list.txt` with the cleaned-up list.

```
# Convert the updated list back to a string, each IP on a new line
with open(import_file, "w") as file:
    file.write("\n".join(ip_addresses))
```

Outcome

This script ensures that any unauthorized or outdated IP addresses are promptly removed from the access list, improving the overall security of our internal network. It reduces the risk of human error and makes it easier to manage changes in access control in a consistent, auditable way.